MODULE 07 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

while loops

Counting passes, tracing with a table, loops that never end, break, continue, and the else clause.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 07 of 28

admin@sthithapragna.com

A while loop tests its condition **before every pass**, including the first. If the condition starts false, the body never runs at all.

Test, run the body, go back and test again

```
1  count = 1                # 1. set up
2  while count <= 3:        # 2. test
3      print(count)
4      count += 1           # 3. update
5  print("done")            # after the loop
```

1. Set up the variable the condition depends on.
2. The condition is tested before every pass. The colon and indentation work as for if.
4. Change something the condition depends on, or it can never become false.

Output: 1, 2, 3 on separate lines, then done.

Tracing the loop, one row per test

COUNT	COUNT <= 3	OUTPUT
1	True	1
2	True	2
3	True	3
4	False: loop ends	

EXAM TECHNIQUE

One row for every time the condition is tested. The body ran once for every True: three passes.

How many passes? It depends on the start and the test

START	CONDITION	VALUES SEEN	PASSES
`i = 0`	i < 3	0, 1, 2	3
`i = 0`	i <= 3	0, 1, 2, 3	4
`i = 1`	i < 3	1, 2	2
`i = 1`	i <= 3	1, 2, 3	3

THE SHORTCUT

Counting up by 1: with <, passes = stop − start. With <=, add one.

When the counter does not step by one, trace it

```
1  x = 20
2  while x > 0:
3      print("*")
4      x //= 2            # 20, 10, 5, 2, 1, then 0
5  floor = 1
6  while floor <= 10:
7      floor += floor     # 2, 4, 8, 16: four passes
```

1-4 x takes 20, 10, 5, 2, 1: five stars. Halving 1 gives 0, which ends the loop.

5-7 floor doubles: 1, 2, 4, 8 each pass the test. At 16 the test fails.

Four ways to write an infinite loop by accident

CODE	WHY IT NEVER ENDS
`while i < 5: print(i)`	Nothing in the body changes i.
`while speed < 30: speed *= 2`	speed starts at 0, and 0 doubled is still 0.
`while i != 10: i += 2`	i starts at 1: 1, 3 … 9, 11. It skips 10.
`while i < i + 2: i += 1`	Both sides grow together. Always true.

THE TEST

Ask of every loop: does the body move the variable towards making the test false, and can it land there?

break leaves the loop immediately

```
1  n = 0
2  while True:
3      n += 1
4      if n == 3:
5          break
6      print(n)
7  print("after", n)       # 1, 2, then after 3
```

2 while True never ends by itself. Something inside must break out.

5 break skips the rest of the body, and the rest of the loop, at once.

break and continue are only legal inside a loop. Anywhere else, they are SyntaxErrors.

continue skips to the next test

```
1  x = 0
2  while x < 6:
3      x += 1
4      if x % 2 == 0:
5          continue
6      print(x)              # 1, 3, 5
```

5 continue abandons this pass and jumps straight back to the condition.

3 The update comes before continue, so it always happens.

Update first, or continue can skip it forever

UPDATE AFTER CONTINUE

Never ends

```
1  x = 0
2  while x < 6:
3      if x % 2 == 0:
4          continue
5      x += 1
```

x is 0, which is even. continue skips the update, so x is 0 forever.

UPDATE BEFORE CONTINUE

Ends after six passes

```
1  x = 0
2  while x < 6:
3      x += 1
4      if x % 2 == 0:
5          continue
```

The update always runs, so x reaches 6 and the test fails.

THE TRAP

If continue can fire before the update line, check whether it can fire forever.

else runs when the condition turns false

```
1  i = 1
2  while i <= 3:
3      i += 1
4  else:
5      print("else runs, i is", i)     # i is 4
```

4 The else lines up with while. It runs once, when the test is false.

2-5 A break inside the loop would skip the else completely.

break is the only thing that skips the else

LOOP ENDS NORMALLY

else runs

```
1  n = 0
2  while n < 3:
3      n += 1
4  else:
5      print("no break")
```

The test fails at n == 3, so no break was needed: the else runs.

LOOP ENDS WITH BREAK

else is skipped

```
1  n = 0
2  while n < 3:
3      n += 1
4      if n == 2:
5          break
6  else:
7      print("no break")
```

break fires at n == 2. The else is skipped, and nothing prints.

ON THE EXAM

A loop's else means no break happened. It has nothing to do with whether the body ran.

A loop that never runs still runs its else

```
1  count = 10
2  while count < 5:
3      print("never")
4  else:
5      print("zero passes, else still runs")
```

2 The very first test is false, so the body runs zero times.

4-5 No break happened, so the else runs.

The sentinel pattern: stop on a special value

```
1  total = 0
2  value = int(input())
3  while value != 0:
4      total += value
5      value = int(input())
6  print(total)
```

2 Read once before the loop, so the first test has a value to check.

5 Read again at the end of the body. The special value 0 ends the loop.

If the user types 4, 5, then 0, the program prints 9. The 0 is never added.

while code that never starts

CODE	ERROR
`While True:`	SyntaxError. Python 3.14 adds: Did you mean 'while'?
`while x < 5`	SyntaxError: expected ':'
`break outside a loop`	SyntaxError: 'break' outside loop
`continue outside a loop`	SyntaxError: 'continue' not properly in loop

> ON THE EXAM
>
> Python has no do-while loop. To run a body at least once, use while True with a break.

What to carry into the exam

- The condition is tested **before every pass**. False at the start means zero passes.
- Count passes with a **trace table**: one row per test, passes = number of True rows.
- break leaves the loop; continue jumps back to the test. Update before continue.
- A loop's **else** runs unless a break fired, even after zero passes.
- Infinite: no update, an update that cannot change the value, or one that skips the stop value.

How many stars does this program print?

```
1  x = 20
2  while x > 1:
3      print("*")
4      x //= 3
```

A Two

B Four

C Three

D The loop never ends

◆ PAUSE AND DECIDE

How many stars does this program print?

```
x = 20
while x > 1:
    print("*")
    x //= 3
```

x: 20 (star), 6 (star), 2 (star), 0 (test fails).

A Two

Stops at x == 2, but 2 > 1 is still true.

B Four

Counts the final test as a pass. At 0 the body does not run.

C Three

20, 6 and 2 pass the test. 2 // 3 is 0, which fails it.

D The loop never ends

Floor division by 3 keeps shrinking x until it reaches 0.

THE SIGNAL

Write each value of the variable at the moment of the test. Count only the tests that were True.

What is the output of this program?

```
1  i = 0
2  while i < 5:
3      i += 1
4      if i == 3:
5          continue
6      print(i, end=" ")
```

A `1 2 4 5`

B `1 2`

C `0 1 2 4`

D `1 2 3 4 5`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  i = 0
2  while i < 5:
3      i += 1
4      if i == 3:
5          continue
6      print(i, end=" ")
```

i is updated before the print, so the values printed are 1 to 5, minus 3.

A `1 2 4 5`

The update comes first; continue skips only the print for 3.

B `1 2`

Treats continue as break. The loop carries on after 3.

C `0 1 2 4`

Prints before the update. Here the update comes first.

D `1 2 3 4 5`

Ignores the continue. The print is skipped when i is 3.

THE SIGNAL

continue skips the rest of this pass only. break would end the loop.

What is the output of this program?

```
1  n = 10
2  while n > 0:
3      n -= 4
4      if n < 0:
5          break
6  else:
7      print("A")
8  print(n)
```

A. A, then -2

B. 2

C. -2

D. A, then 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 10
2  while n > 0:
3      n -= 4
4      if n < 0:
5          break
6  else:
7      print("A")
8  print(n)
```

n: 10 -> 6 -> 2 -> -2, and -2 < 0 fires the break.

A. A, then -2
The loop ended with break, so the else is skipped.

B. 2
Stops tracing at 2. 2 > 0 is true, so there is another pass.

C. -2
n reaches -2 inside the loop, break fires, the else is skipped.

D. A, then 2
Assumes the test failed at 2. It passed, and break ended the loop.

THE SIGNAL

Ask how the loop ended. Test failed: else runs. break: else skipped.

What is the output of this program?

```
1  i = 1
2  while i != 10:
3      i += 2
4  print(i)
```

A 10

B 11

C 9

D Nothing: the loop never ends

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  i = 1
2  while i != 10:
3      i += 2
4  print(i)
```

i: 1, 3, 5, 7, 9, 11, 13 … always odd, never 10.

A 10

i is always odd, so it can never equal 10.

B 11

i passes 10 without stopping; != 10 stays true for 11.

C 9

9 != 10 is true, so the loop continues past 9.

D Nothing: the loop never ends

Starting odd and adding 2 skips 10 for ever.

THE SIGNAL

With != as the test, check the stop value can actually be reached, not just passed.

What is the output of this program?

```
1  count = 5
2  while count < 5:
3      print("loop")
4      count += 1
5  else:
6      print("else")
7  print(count)
```

A else, then 5

B loop, else, then 6

C 5 only

D loop, then 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  count = 5
2  while count < 5:
3      print("loop")
4      count += 1
5  else:
6      print("else")
7  print(count)
```

5 < 5 is False at the very first test: zero passes, no break.

A. else, then 5

Zero passes, no break: the else runs, and count is unchanged.

B. loop, else, then 6

Runs the body once. The test is checked before the first pass.

C. 5 only

Skips the else because the body never ran. That is not the rule.

D. loop, then 6

Runs the body, and drops the else. Both are wrong.

THE SIGNAL

A loop's else runs whenever no break fired, including after zero passes.

The user types 3, then 4, then -1, pressing Enter after each.

What is the output of this program?

```
1  total = 0
2  count = 0
3  n = int(input())
4  while n >= 0:
5      total += n
6      count += 1
7      n = int(input())
8  print(total, count)
```

A 6 3

B 7 2

C 7 3

D 0 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  total = 0
2  count = 0
3  n = int(input())
4  while n >= 0:
5      total += n
6      count += 1
7      n = int(input())
8  print(total, count)
```

n = 3: pass. n = 4: pass. n = -1: test fails, never added or counted.

A `6 3`

Adds and counts the -1. The test stops the loop before that.

B `7 2`

Two passes: 3 and 4. The sentinel ends the loop at the test.

C `7 3`

Counts the sentinel as a pass. It only reaches the test.

D `0 0`

The first value, 3, passes the test, so the body runs.

THE SIGNAL

In a sentinel loop, the stop value is read and tested, but never processed.